PROJECT BATTLE BOTS

Development & Implementation Collaboration

Between:
Group IT-1B And Teachers

DRAFT 12.02.2026

Technical Design

Group IT – 1B

Ivan Lykhota
Friso Terpsta
Joel Shoniwa
Muhammad Umar Kamran
Kyra Kovacs
Jehan Master

1.0 GENERAL INFORMATION 3
1.1 Purpose 3
1.2 Scope 3
1.3 Project References 4
1.4 Acronyms and/or Definitions 4
1.5 Points of Contact 5
1.5.1 Information and Coordination 5
1.5.2 Roles and Responsibilities 5
1.5.3 Administrative support and Oversight 5
2.0 Hardware Components 5
2.1 Core intelligence and structure 5
2.2 Movement and Actions 6
2.3 Visuals and Interaction 7
3.0 Functional Requirements and Impact 9
3.1 Summary of Functions, Functional Requirements 9
3.2 Summary of Impacts 10
3.2.2 Operational impacts 10
4.0 Performance Requirements 12
4.1 Specific Performance Requirements 12
4.1.1 Accuracy and Validity 12
4.1.2 Timing and Capacity 12
4.1.3 Failure Contingencies 12
5.0 Additional System Requirements 13
5.1 System Description 13
5.2 System integration 13
5.3 Customization and Flexibility 14
5.4 Sustainability and Open-Source Plans 14
5.5 System Documentation 14
5.6 Rights to Code and Data 15
5.7 Configurable System Parameters 15

5.8 System Development and Go-live Approval Process 15
6.0 Equipment and Software 17
6.1 Equipment 17
6.2 Software 17

1.0 GENERAL INFORMATION

1.1 Purpose

The purpose of this document is to provide sufficient information to stakeholders and evaluators to allow their approval and support of the development, programming and testing of a BattleBot system. This document outlines the high-level technical and functional requirements of the project and describes the roles and responsibilities of the six-member team involved in its execution. It details the coordination required to successfully code and set up robots wiring, including software development, system integration, testing, and maintenance responsibilities. The document also provides an overview of the resources and estimated effort required to complete and sustain the project.

1.2 Scope

This Functional and Technical Requirements Document outlines the functional, performance, hardware, software, safety, and system requirements for the BattleBot project as defined by the project team. The proposed solution consists of the design, set up robots wiring, programming, testing, and operation of three autonomous BattleBot robots developed by a six-person team.

The BattleBot system will:

- Include three autonomous robots built using provided and approved hardware components such as a microcontroller (Arduino), motors, motor drivers, sensors, and power supply units.
- Integrate hardware components including distance sensors, line sensors, servo motors, and drive motors to enable navigation, obstacle detection, and task execution.
- Implement embedded software (C++ using the Arduino platform) to control and coordinate hardware components, sensors, and actuators.
- Support safe and reliable operation through collision avoidance, sensor validation, and hardware protection mechanisms.
- Allow team members to collaboratively develop, test, and maintain both hardware configurations and software using documented standards and version control.

- Enable structured testing of hardware–software interaction to verify correct behaviour under competition conditions.
- Define roles and responsibilities within the six-person team for hardware assembly, software development, testing, and documentation.

The scope of this work includes the initial hardware assembly, software development, system integration, and testing of the three BattleBot robots during the project period. Future enhancements, such as additional hardware modules, expanded robot functionality, or alternative competition modes, are outside the current scope but may be supported by the system design.

1.3 Project References

Key documents supporting this proposal are listed below as reference:

- Action Plan
- Requirement analysis
- Module book

1.4 Acronyms and/or Definitions

Arduino	Open-source microcontroller platform used to control the BattleBot hardware and execute embedded software.
C/C++	Free and open-source version control system used by the team to manage and track changes in source code.
IDE	Integrated Development Environment; software application (e.g., Arduino IDE) used. for writing, compiling, and uploading code to the robot.

I/O (Input/Output)	Communication between the microcontroller and external devices such as sensors (input) and actuators (output).
Version Control	System that records changes to files over time, allowing multiple team members to collaborate effectively.

1.5 Points of Contact

1.5.1 Information and Coordination

Points of contact relevant to this project are listed in the introductory section of this document. Once the project has received approval from the supervising teacher and/or project coordinator, this document will serve as a formal agreement outlining the agreed-upon objectives, responsibilities, and requirements of the BattleBot project. Each project team member will be required to acknowledge their role and responsibilities within the team, confirming their acceptance of the tasks and expectations defined in this document.

1.5.2 Roles and Responsibilities

Subgroup 1 (Kyra and Jehan)	Programming, setup wiring, testing the robot which needs to complete line following task.
Subgroup 2 (Joel and Friso)	Programming, setup wiring, testing the robot which needs to complete line maze task.
Subgroup 3 (Ivan and Muhammad Umar)	Programming, setup wiring, testing the robot which needs to complete line maze task.

Project team members who participate in the BattleBot project commit to the following:

- Team members will contribute to the hardware assembly and programming of their assigned BattleBot within the agreed project timeline (8 weeks).
- Team members agree to have internal team processes in place to ensure that code, hardware configurations, and documentation are maintained and kept up to date.
    - Updates must be made promptly after a change, including:
        - Modifications to hardware components or wiring.
        - Changes to sensor configurations or motor control logic.
        - Updates to software functionality or behaviour.
- Team members who do not adhere to agreed development, testing, or documentation standards may be reassigned tasks or required to make corrections, as determined by the team lead or supervising teacher.
- Team members agree to use the shared code repository and documentation as the single source of truth and will not request or maintain duplicate or conflicting versions outside the agreed tools.
- Team members agree to participate in all ateliers, scheduled testing sessions, demonstrations, and evaluations, and to provide required input or data related to robot performance.
- Team members who integrate shared code modules agree to synchronize their local development environment regularly to ensure compatibility and consistency across all three robots.

1.5.3 Administrative support and Oversight

The project team lead and teachers will operate as the administrative support and oversight group for the BattleBot project, performing the following tasks:

- Monitor project progress, including code commits, task completion, testing results, and overall team participation.
- Maintain awareness of project needs related to hardware changes, software improvements, or scope adjustments.
- Coordinate communication among team members and provide regular progress updates and feedback.
- Receive, document, and track ongoing feedback from team members and teachers.
- Coordinate periodic reviews of the entire project, including testing phases and final evaluation.

2.0 Hardware Components

2.1 Core intelligence and structure

- The **Arduino NANO microcontroller**, which is the brain of the battle bot, executes the code that is written in the Arduino application. This is what tells the battle bot what to do.

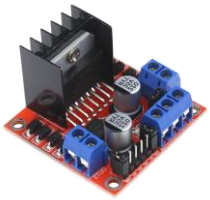
- The **PCB with Uno Footprint** is a custom board that allows the nano to connect to the shields (the add-on boards) made for the larger Arduino Uno, which makes it easier to stack hardware.

- **The Metal chassis** is what gives the frame the ability to mount the motors, wheels, and electronics used for its durability.

- **The power bank (10.000 mAh)** provides the power to the robot so that it can complete its task once the code is uploaded, while it's not connected to the device that is programming it.

- The bot has two on and OFF switches for the logic circuit and the motors. The motor switch is what powers the motors, it also stops the bot from moving when you are programming it.

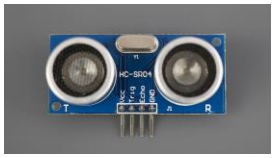
- **The motor driver** is the translator; it takes the Nano's low-power signals, then uses the power bank's energy to the motors to control the speed and direction because the motors need high-power current.



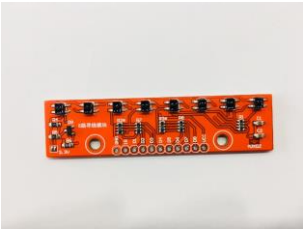
2.2 Movement and Actions

- Two **electric motors** convert electrical energy to mechanical energy to turn the wheels. The gripper with a servo motor moves to a specific angle this allows the gripper to open and close the exact amount of an object.

- The **two rotation sensors** on the bot sit on the monitor and calculates counts the ticks as the wheels move this then allows the Nano to calculate the distance that the bot has travelled.
- The **ultrasonic sensor** are the eyes of the bot they emit ultrasonic "ping" and measures the time it takes to echo back, which then allows it to calculate the distance the bot is from the object so that it can avoid crashing into said object.



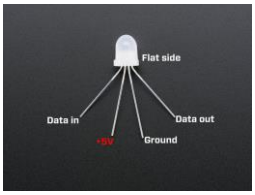
- The **eight analogue line follower sensors** are infrared, which detect the difference between a dark line and a light floor. Since the sensor has eight of them, this allows the bot to know exactly how far of centre it is.



- The **ESP01 Module** allows the bot to connect to the internet or a local router so you can control the bot from a web browser or send data to a cloud dashboard
- The **Bluetooth HC-05** is used for short range wireless connection; it lets you connect to a phone and use it as a remote controller.

2.3 Visuals and Interaction

- **4 Neo pixels** are the smart lights they are RGB LEDs that can be chained together you can make them flash certain colours to represent lights in a car like turns signals which is used by one wire to the Nano.



- There are **three Push buttons and three LEDs**, the user interface where you can select a mode like manual control or like line following or a start and stop system like the traffic lights.

- And the **mini breadboard** is the testing area, this small grid is where you plug in the extra components with the pins without soldering, mainly used for temporary trials.

3.0 Functional Requirements and Impact

3.1 Summary of Functions, Functional Requirements

- **Data Capture**
    - ♣ Arduino Api is based on C++

    - *Types of data*
        - ♣ Distance readings
        - ♣ Line tracking values
        - ♣ Gripper value

    - *Storage needs*
        - ♣ 32KB (2KB used by bootloader)
        - ♣ Data is sent over serial communication

    - *Power Supply*
        - ♣ Via USB or external power supply

    - *Analog Input pins*
        - ♣ 8

    - *Board*
        - ♣ Arduino Nano

    - *Processor*
        - ♣ ATmega328P or ATmega328P (Old Bootloader)

- **Preferred Data Capture Method**
    - *Sensor types required*
        - ♣ Ultrasonic distance sensor
        - ♣ Analogue line sensor

- *Input methods*
    - USB serial connection for configuration

- **User Interface Layout**
    - Power buttons

- **Output Requirements**
    - *Visual Outputs*
        - LED power status indicators
        - NeoPixel LED
    - *Data outputs*
        - Serial output for debugging
    - *Mechanical outputs*
        - Motor actions
        - Servo movements

3.2 Summary of Impacts

3.2.1 Organizational impacts

The project will require 3 subgroups in our team working on separate assignments. These roles have been filled in on the Action Plan on a voluntary basis. The project will require a team leader, a quality assurance person, a minutes secretary, programmers and electromechanical assemblers. Finances won't be calculated into this project.

The quality assurance person will be a person on our team as is visible in the Action Plan. While the project is ongoing, they will make sure weekly that the robot's electrical wiring, the code and every other detail is up to standards that are written in this document. Any other requirements will be assigned by the client(s) and the team leader will make sure we can make it by the end of the deadline, and the quality assurance person will make sure the added requirements will be up to standard.

All people associated with this project (client(s) and team members), the publication of the finished project will be approved by the quality assurance person and the team leader.

3.2.2 Operational impacts

3.2.3 Project Working Group Impacts

4.0 Performance Requirements

4.1 Specific Performance Requirements

4.1.1 Accuracy and Validity

The system will employ numerous data quality assurance techniques, including but not limited to:

- Sensor Calibration:
  Ultrasonic/line sensors must be calibrated and tested to ensure that they're receiving the expected signals
- Line detection:
  The *robot* must identify the track under normal light conditions (up to at least 90% accuracy).
- Object detection:
  The *robot* must correctly identify the obstacles/objects under normal light conditions and *avoid them*

4.1.2 Timing and Capacity

- The system is intended to be available 24 hours per day, 365 days per year with the exception of scheduled and pre-notified maintenance, if needed.
- Code will become immediately available for use when new features are added to the system.
-

4.1.3 Failure Contingencies

- *The system is reliant on pre-input code therefore is noncritical*
- If the robot loses track for more than 1 second, it must stop and recalibrate. In the case that recalibration is unsuccessful the robot must be reset and manually placed again on the track.
- The robot must stop immediately when an obstacle in detected within *cm to avoid collision.
- When battery voltage drops below the expected value, the robot must reduce its speed or come to a stop to avoid unpredictable behaviour.
- If a motor fails to respond the system should come to an immediate stop to avoid uncontrolled behaviour.

5.0 Additional System Requirements

5.1 System Description

The Battle Bot is an already made and developed system built around an Arduino Nano. The robot needs to be designed and coded to follow a line, detect obstacles and avoid line obstacles.

The Batlle Bot has five functions

1. Control system = (Arduino Nano)
2. Drive system = DC motors + L298N motor driver
3. Sensor system = Line array sensor, ultrasonic distance sensor and rotation sensor.
4. Actuator system = servo motor (gripper)
5. Power system = rechargeable battery pack

5.2 System integration

Component	**Arduino Pin**	**Type**
Ultrasonic Trigger	D4	Digital Output
Ultrasonic Echo	D7	Digital Input
Line Sensor	A0–A4 and A6/A7(Analog only)	Analog Input
Motor Driver	D3, D5, D,6, D9, D10, D11	Digital Output
Servo Motor	D9	PWM
Rotation Sensor	D2	Interrupt

Pin Selection

PWM-pins are used for motors and servo to enable speed and have more voltage. Analog pins are used for the lines sensors for accurate measurement of reflected light. Interrupt pins are used for wheel encoders for pulse detection.

Power

The Battle Bot is powered by a rechargeable battery pack. The motor driver is supplied directly from the battery. The Arduino Nano and sensors are powered with 5V.

Signal Flow

Sensor data is continuously read by the Arduino Nano. It processes the date and determines the date, then the control signals are sent to the motor driver and servo motor to execute the required movement.

5.3 Customization and Flexibility

The Batlle Bot is designed in a very open and easily changeable design All sensors and actuators are connected using detachable jumper wires, allowing easy replacement and if needed upgrades.

The Battle Bot is structured into a separate functional module: Line following, Obstacle detection, Motor control, Servo control.

This structure makes the addition of new features easy without making huge changes to the existing code.

5.4 Sustainability and Open-Source Plans

The Battle Bot is developed using open-source software used from the Arduino platform. The Arduino platform (website: Arduino.cc) is freely available and widely support

All project files, source code and extra information code, are stored in a shared version-control repository. This allows further development, maintenance, and reuse in future projects.

Making this rule lets us keep track of the code and of course to see what was committed at what time by who.

5.5 System Documentation

We try to have helpful comments at all the important lines of code to remember for them self and help others understand. Especially for functions, commenting on a description of what it does.

And a more documentation will be in needed for this project:

- Complete wiring diagram
- Pin configuration (I/O list)
- System architecture diagram
- Hardware component overview
- Calibration and setup instructions
- User guide for operation and maintenance

5.6 Rights to Code and Data

All software, documentation and test results made during the Battle Bot is property of the project group (IT 1b) and the educational institution. The source code and design documents may be reused for educational and research purposes.

All measurement data and test results are stored in the project portfolio and are used only for project changes and quality improvement.

No information should be sent or shared with other project teams, and ownership will always stay with the project group.

5.7 Configurable System Parameters

All system parameters and configured data are calibrated to make the Batlle Bot work correctly.

Here are some examples for parameters and configurations:

- Maximum motor speed
- Line-following correction values
- Obstacle detection distance.
- Servo rotation limits
- Acceleration and deceleration rates
- Sensor filtering values

5.8 System Development and Go-live Approval Process

Developments Phases:

First, we check everything, like power, motors, sensors, control, ect. Learning how to code what software we use and getting familiar with the robot.

Second is using the robots and making assignments with them, learning how the robot works, and adjusting values with code to make it work correctly.

Third is integrating the robot to follow the mazes (end race).

Lastly, the last check before the race, testing it multiple times, checking for crashes, and making sure the battery is full (important).

6.0 Equipment and Software

6.1 Equipment

The BattleBot system is built around an Arduino Nano microcontroller mounted on the BattleBot V0.3 base board. The robot consists of a mobile chassis equipped and it have two DC motors for movement and servo motor which is used for the gripper mechanism. The movement of motor is handled through the motor driver which is integrated on the BattleBot board.

The robot does the environmental sensing using an ultrasonic distance sensor for obstacle detection and it used a line sensor array for line-following task. The supplying of power is done via an onboard battery pack. All components are connected by using jumper wires and standard connectors provided with the BattleBot kit.

Testing and development are performed using a laptop connected to the Arduino via USB. Simple measurement tools such as a ruler and stopwatch are used during calibration and testing. Other than that, no additional hardware outside the provided BattleBot kit is required.

6.2 Software

The BattleBot is programmed using the Arduino Legacy IDE (version 1.8.x). Software is written in C/C++ using the Arduino framework. Arduino IDE is also used for version control during development to test the program and to debug the robot with the help of the serial number.

Version control is managed by using Git to track changes and support collaboration within the project group. Hardware schematics and wiring overviews are created using Fritzing or equivalent diagramming software. During the development, standard Arduino and C++ coding component are followed, in accordance with the guidelines provided in the module.